

Assignment 1: The Turtlebot

Intelligent Robotics course 2025/2026

Erba Lorenzo - lorenzo.erba.1@studenti.unipd.it

Fusari Nicolò - nicolo.fusari@studenti.unipd.it

Piccoli Leonardo Arduino - leonardoarduino.piccoli@studenti.unipd.it

June 1st 2026

1 Introduction

This project consists of the implementation of an autonomous robotic system capable of exploring an indoor environment by detecting AprilTags placed in a room and navigating toward the corresponding targets.

The software framework provided for this assignment is composed of a complete simulation environment built on ROS 2, Nav2 and Gazebo, which provides the robot model, the virtual world and launch files.

GitHub repository: github.com/lorenzoerba1/group11_assignment_1

Video link: youtube.com

The simulation reproduces the IAS laboratory containing some tables. The robot's objective is to detect the tags through an external camera, autonomously reach the position between them and finally detect the three tables while returning their relative positions.

To solve the assignment, the system was organized into a modular architecture composed of several ROS 2 nodes, visible in fig.1, each responsible for a specific task.

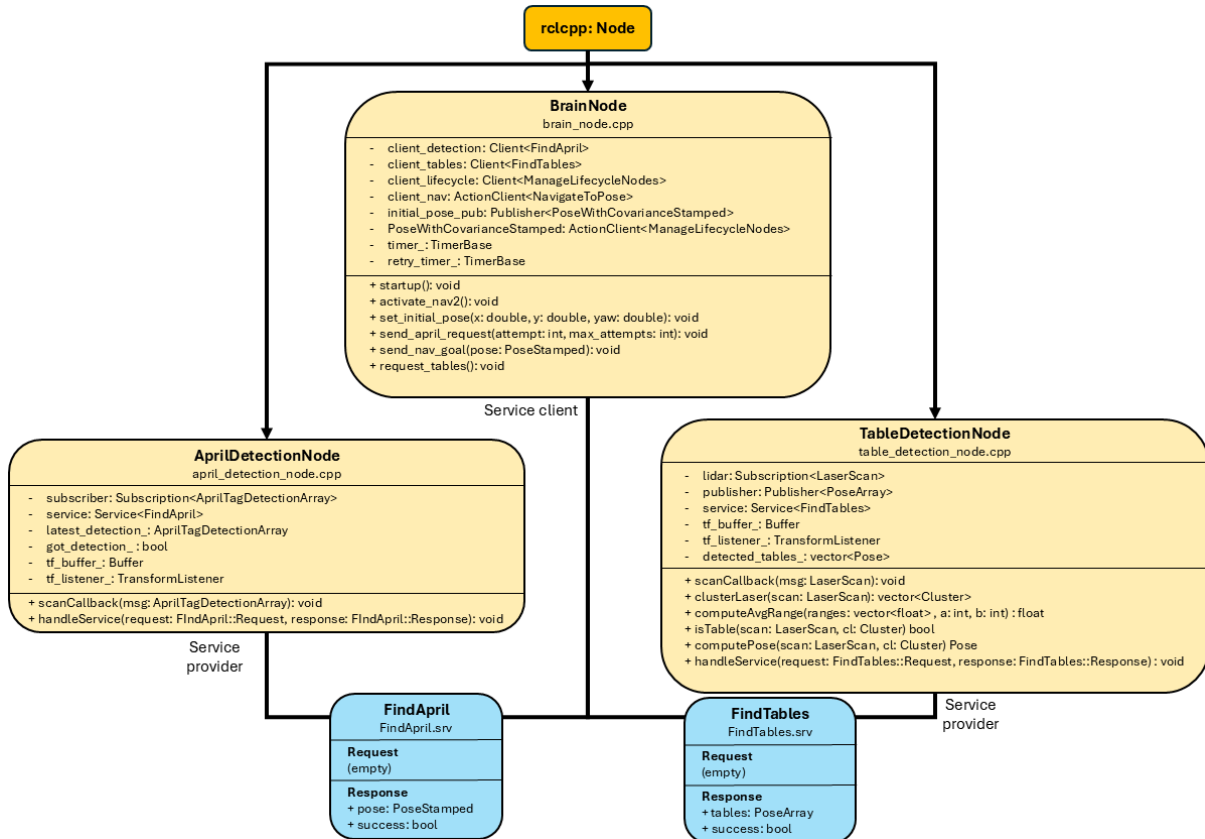


Figure 1: UML diagram

2 Brain node

It is the component of the architecture which coordinates the overall behavior of the robot by interacting with the other functional modules through services and actions.

The node is implemented in `brain_node.cpp` as a simple state-machine, where each phase is executed only after the successful completion of the previous one. The workflow consists of three main stages: getting the pose of the AprilTags, then triggering the robot navigation toward the computed target position and finally requesting the detection and localization of the tables.

At startup six communication interfaces are created:

- a service client for `find_april`, used to request the target pose computed between the two AprilTags,
- two service clients for the Nav2 lifecycle managers, `/lifecycle_manager_localization` and `/lifecycle_manager_navigation` (both `/manage_nodes`),
- a publisher on `/initialpose`, to initialize AMCL with the robot starting pose
- an action client for `navigate_to_pose`, standard Nav2 action server,
- a service client for `find_tables`, necessary to obtain the final positions of the tables.

3 Apriltag detection

This part of the system is responsible for detecting the AprilTags placed in the environment and computing the target position associated with them. This functionality is implemented in the `april_detection_node.cpp` node, which acts as a ROS 2 service server and provides the `find_april` service.

The package `apriltag_ros`, from the provided framework, processes the images acquired by the external camera and publishes both a detection topic (`/apriltag/detections`) and a set of TF transforms, one for each detected tag. Each transform represents the pose of a tag relative to the camera reference frame.

The *AprilDetection* node subscribes to the topic `/apriltag/detections`, which contains the list of tags currently visible to the camera. The content of this topic is not used directly for pose estimation, instead, it is employed as a flag to verify that at least one detection has been received before processing a service request.

When the `find_april` service is called from the *Brain* node, the component performs the following steps: first it checks whether AprilTag detections are available and defines their identifiers, then it retrieves from the TF tree the transforms associated with the two tags to extract their 3D positions. After it has obtained the transforms it can calculate the coordinates of the mid-point transforming it in the robot reference frame and returning the computed pose through the service response.

The communication between the *Brain* node and the *AprilTag Detection* node is implemented through the custom service `FindApril`, because the target position is required only once before the navigation is started. This service does not contain input fields and is defined as:

- `geometry_msgs/PoseStamped pose` - the estimated target pose,
- `bool success` - a flag indicating whether the computation was successful.

4 Navigation

The navigation phase is managed directly in `brain_node.cpp` rather than delegating this task to a dedicated *Navigation* node, to not increase the complexity of the architecture too much. Firstly the two service clients for the Nav2 lifecycle managers are started, one for Localization and one for Navigation, using the `nav2_msgs/srv/ManageLifecycleNodes` interface to allow the node to activate the Nav2 components programmatically.

Once the localization stack is active, the initial pose is published to the `/initialpose` topic using a `geometry_msgs/PoseWithCovarianceStamped` message; the pose is expressed in the `map` frame and includes both position and covariance information.

Then the node waits until the Nav2 action server: `navigate_to_pose` becomes available, the action client uses the standard Nav2 interface `nav2_msgs/action/NavigateToPose`. The target pose is obtained from the custom service `find_april`, returning the midpoint between the two AprilTags. If the pose is valid, it is forwarded directly to Nav2 as a `NavigateToPose::Goal` message containing: the target position and orientation, the `map` frame identifier and the current timestamp. It also registers the action callbacks used to monitor the navigation process.

When the robot has reached the desired destination, the callback reports a successful execution, so the *Brain* node can proceed to find the tables.

5 Tables detection and localization

The final phase of the assignment consists in detecting the tables present in the environment and estimating their positions with respect to the robot, all of this is encoded in `table_detection_node.cpp`. The node continuously subscribes to the `/scan` topic, where the LiDAR measurements are published and processes each scan to identify potential tables. The detected poses are then stored internally and published on the `/tables` topic as a `geometry_msgs/PoseArray`.

Each `LaserScan` message contains a sequence of range measurements acquired by the LiDAR sensor. The scan is first segmented into clusters by grouping consecutive points whose distance difference is smaller than a threshold of 0.15 m. The hardest part to accomplish in this project was finding exactly the clusters that indicate the tables on the map, without getting confused by the walls or random noises.

To determine whether a cluster corresponds to a table, three points are considered: the first, the middle and the last. If the middle point is closer to the robot than both the other two then the cluster is classified as table, due to its convex profile which differs from a planar wall.

To further improve the stability of the detection and reduce the false positives, a filtering on invalid LiDAR measurements was introduced together with min/max cluster sizes.

The initial estimate is expressed in `base_scan` corresponding to the LiDAR reference frame, so the poses have to be transformed into the `odom` frame with TF2, to remain constant while the robot moves.

TableDetection exposes the `FindTables` service, which allows the *Brain* node to request the most recent set of detected tables; the service, without any input field is defined as:

- `geometry_msgs/PoseArray tables` - the poses of all detected tables,
- `bool success` - a flag indicating whether the computation was successful.

The *Brain* node then iterates on all returned poses and prints the coordinates for each table.